GRAPHQL VỚI RUBY ON RAILS

Bước 35 — Fragments

Giáo án 1:1 • Project Management API • Client Request

1. Fragment chỉ ở GraphQL hay request của client?

Fragment là cú pháp của GraphQL và thường được viết ở phía client trong GraphQL document/request. Rails không có một Fragment Resolver riêng.

```
Fragment
→ thuộc GraphQL language
→ client vi□t trong query/mutation
→ server nhận document có fragment
→ GraphQL engine validate/expand fragment
→ resolver x□ lý các field c□n resolve
```

2. Không dùng Fragment

```
query GetTasks($projectId: ID!) {
  tasks(projectId: $projectId, first: 10) {
    nodes {
      id
      title
      status
      priority
      dueDate
    }
  }
}
```

Nếu mutation cũng cần đúng các field đó, bạn phải lặp lại selection set.

3. Dùng Fragment

```
fragment TaskFields on Task {
  id
  title
  status
  priority
  dueDate
}

query GetTasks($projectId: ID!) {
  tasks(projectId: $projectId, first: 10) {
    nodes {
      ...TaskFields
    }
  }
}
```

...TaskFields là fragment spread. Nó tái sử dụng selection set.

4. Fragment nằm ở đâu?

```
src/
└── graphql/
    ├── fragments/
    │   └── taskFields.js
    ├── queries/
    │   └── getTasks.js
    └── mutations/
        └── createTask.js
```

```
export const TASK_FIELDS = `
  fragment TaskFields on Task {
    id
    title
    status
    priority
    dueDate
  }
`;
```

Với Axios/plain string, fragment phải được ghép vào GraphQL document gửi lên server.

5. Axios request thực tế

```
const response = await axios.post(
  "/graphql",
  {
    query: `
      fragment TaskFields on Task {
        id
        title
        status
        priority
        dueDate
      }

      query GetTasks($projectId: ID!) {
        tasks(projectId: $projectId, first: 10) {
          nodes {
            ...TaskFields
          }
        }
      }
    `,
    variables: {
      projectId: "10"
    },
    operationName: "GetTasks"
  },
  {
    withCredentials: true
  }
);
```

HTTP body chứa query document, variables và operationName. Fragment nằm bên trong query document.

6. Rails có cần TaskFieldsFragment không?

Không. Không tạo class TaskFieldsFragment và không tạo resolver task_fields.

```
class TaskType < Types::BaseObject
  field :id, ID, null: false
  field :title, String, null: false
  field :status, Types::Enums::TaskStatusEnum, null: false
  field :priority, Types::Enums::TaskPriorityEnum, null: false
  field :due_date, GraphQL::Types::ISO8601Date, null: true
end
```

Rails chỉ cần GraphQL type có các field mà fragment sử dụng.

7. `on Task` nghĩa là gì?

```
fragment TaskFields on Task {
  id
  title
}
```

on Task nghĩa là fragment này áp dụng cho GraphQL type Task. Không thể spread TaskFields vào Project.

8. Fragment có thể chứa Fragment khác

```
fragment UserFields on User {
  id
  email
}

fragment TaskFields on Task {
  id
  title
  assignee {
    ...UserFields
```

```
  }
}
```

9. Fragment dùng cho Query và Mutation

```
query GetTask($id: ID!) {
  task(id: $id) {
    ...TaskFields
  }
}

mutation UpdateTask($input: UpdateTaskInput!) {
  updateTask(input: $input) {
    task {
      ...TaskFields
    }
    errors
  }
}
```

10. Fragment có giảm SQL không?

Không tự động. Fragment chỉ tổ chức GraphQL selection set. Nó không tự preload association, không giải quyết N+1 và không cache database.

Nếu fragment lấy assignee thì Rails vẫn cần Dataloader/preload phù hợp để tránh N+1.

11. Fragment khác Input / Variable / Operation Name

```
Input       → dữ liệu client G□I VÀO
Fragment    → field client MU□N NHẬN V□
Variable    → giá trị động, ví dụ $projectId
Operation   → tên query/mutation, ví dụ GetTasks
```

12. Naming convention

```
UserFields
ProjectFields
TaskFields

TaskSummaryFields
TaskDetailFields

ProjectSummaryFields
ProjectDetailFields
```

Fragment nên phản ánh nhu cầu UI. Không nên tạo một fragment khổng lồ rồi dùng ở mọi màn hình.

13. Tổ chức frontend chuyên nghiệp

```
src/
├── graphql/
│   ├── fragments/
│   │   ├── taskFragments.js
│   │   └── userFragments.js
│   ├── queries/
│   │   └── getTasks.js
│   └── mutations/
│       ├── createTask.js
│       └── updateTask.js
└── api/
    └── graphqlClient.js
```

```
export const TASK_SUMMARY_FIELDS = `
  fragment TaskSummaryFields on Task {
    id
    title
    status
    priority
    dueDate
  }
```

```
`;

export async function graphqlRequest({
  query,
  variables,
  operationName
}) {
  const response = await axios.post(
    "/graphql",
    { query, variables, operationName },
    { withCredentials: true }
  );

  return response.data;
}
```

14. Ghi nhớ Bước 35

```
Fragment
→ thuộc GraphQL language

Được vi□t □ đâu?
→ thường □ frontend/client

Được g□i □ đâu?
→ trong GraphQL document c□a request

Rails c□n làm gì?
→ không tạo Fragment class
→ ch□ c□n schema/type có field fragment s□ dụng

Fragment dùng đ□ làm gì?
→ reuse selection set

Fragment có gi□m SQL không?
→ không

Fragment dùng cho Query và Mutation?
→ có
```

```
CLIENT
│
├── Operation: GetTasks
├── Variables: projectId
└── Fragment: TaskFields
        │
        ▼
GraphQL document
        │
        ▼
POST /graphql
        │
        ▼
Rails GraphQL Schema
        │
        ▼
validate fragment against TaskType
        │
        ▼
Resolver
        │
        ▼
Response
```